EXP 19: Implementation of Naive Bayes classification for Bank Loan prediction

CODING:

```
# Bank Loan Prediction using Naive Bayes

import math

# Dataset: [Age, Monthly Income (in thousands), Loan Status]
data = [
  [25, 20, "Rejected"],
  [30, 25, "Rejected"],
  [35, 30, "Approved"],
  [40, 50, "Approved"],
  [45, 60, "Approved"],
  [50, 55, "Approved"]
]

classes = ["Approved", "Rejected"]

# Calculate mean
def mean(values):
  return sum(values) / len(values)

# Calculate variance
def variance(values):
  m = mean(values)
  return sum((x - m) ** 2 for x in values) / len(values)
```

```
# Gaussian probability
def gaussian(x, m, v):
  if v == 0:
    v = 0.0001

  return (1 / math.sqrt(2 * math.pi * v)) * math.exp(
    -((x - m) ** 2) / (2 * v)
  )

# Train the model
model = {}

for c in classes:
  rows = [row for row in data if row[2] == c]
  model[c] = []

  for i in range(2):
    values = [row[i] for row in rows]
    model[c].append((mean(values), variance(values)))

# Predict loan status
def predict(age, income):
  sample = [age, income]
  scores = {}

  for c in classes:
    rows = [row for row in data if row[2] == c]
    probability = len(rows) / len(data)
```

```
        for i in range(2):
            m, v = model[c][i]
            probability *= gaussian(sample[i], m, v)

        scores[c] = probability

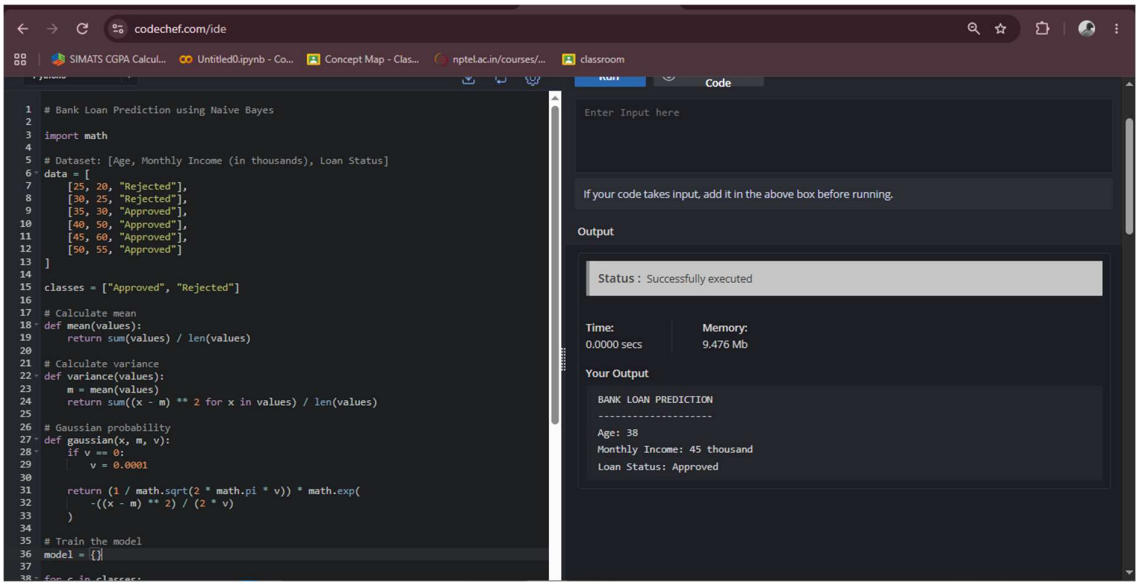
    return max(scores, key=scores.get)

# Test data
age = 38
income = 45

result = predict(age, income)

print("BANK LOAN PREDICTION")
print("--------------------")
print("Age:", age)
print("Monthly Income:", income, "thousand")
print("Loan Status:", result)
```

OUTPUT:




```python
# Bank Loan Prediction using Naive Bayes

import math

# Dataset: [Age, Monthly Income (in thousands), Loan Status]
data = [
    [25, 20, "Rejected"],
    [30, 25, "Rejected"],
    [35, 30, "Approved"],
    [40, 50, "Approved"],
    [45, 60, "Approved"],
    [50, 55, "Approved"]
]

classes = ["Approved", "Rejected"]

# Calculate mean
def mean(values):
    return sum(values) / len(values)

# Calculate variance
def variance(values):
    m = mean(values)
    return sum((x - m) ** 2 for x in values) / len(values)

# Gaussian probability
def gaussian(x, m, v):
    if v == 0:
        v = 0.0001

    return (1 / math.sqrt(2 * math.pi * v)) * math.exp(
        -((x - m) ** 2) / (2 * v)
    )

# Train the model
model = {}
```